

Herald



Protocol Research: gRPC streaming

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	gRPC is a high-throughput binary RPC framework over HTTP/2, with first-class server-streaming + client-streaming + bidirectional-streaming RPCs. For Herald, gRPC complements REST in scenarios where REST's per-request framing overhead dominates (high-rate event ingest from internal systems; large data pulls from cherald). Browser unfriendliness (HTTP/2 trailers) means gRPC is INTERNAL-FACING ONLY in Herald (no public gRPC). Wave 4e.
Issues	open-questions: gRPC-Web bridge for browsers; protobuf schema evolution; gRPC-Gateway REST shim.
Continuation	Wave 4e — open HRD-276..HRD-290; new module <code>commons_grpc/</code> (L1); pherald + cherald + sherald gain gRPC servers.

Constitutional anchors

- **§107 anti-bluff** — gRPC tests use real `grpcurl` invocations + real protobuf-typed clients; assert byte-level binary frame parsing.
- **§11.4.74 catalogue-check** — performed; no `vasic-digital` or `HelixDevelopment` gRPC module. Verdict: depend on `google.golang.org/grpc` + `google.golang.org/protobuf` — both stable, idiomatic; no submodule needed (well-known carve-out per spec V3 §9.1).
- **§11.4.61** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)

- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

gRPC is a high-performance open-source universal RPC framework originated at Google (2015). Wire format: **Protocol Buffers** (binary, schema-based) over **HTTP/2**. License: Apache 2.0.

RPC types. Four:

1. **Unary** — single request → single response. Like REST POST.
2. **Server streaming** — single request → stream of responses. Like SSE.
3. **Client streaming** — stream of requests → single response.
4. **Bidirectional streaming** — independent streams in both directions.

Authentication. Channel-level: SSL/TLS (server + optional client mTLS). Call-level: tokens via metadata (authorization: Bearer <jwt>). The `grpc-auth` packages handle JWT verification on the server side.

Versioning. Protobuf schema evolution rules (reserved fields, additive changes, never reuse numbers).

Adoption. Used by Google Cloud SDK, Kubernetes, etcd, Envoy, CockroachDB, TiDB, ... Pervasive in cloud-native.

Performance. Binary protobuf is ~3-5x more compact than JSON; HTTP/2 multiplexing eliminates head-of-line blocking; streaming saves connection overhead.

Browser unfriendliness. Browsers can't natively speak gRPC (HTTP/2 trailers are blocked by fetch). Workarounds: **gRPC-Web** (separate spec, proxies HTTP/1.1 [?](#) gRPC) or **connect-go** (gRPC-compatible protocol that works over HTTP/1.1).

§2. Specification deep-dive

§2.1 Service definition

```
syntax = "proto3";
package herald.v1;

service Notifier {
  rpc Notify(NotifyRequest) returns
    (NotifyResponse);           // unary
  rpc StreamEvents(EventsFilter) returns (stream
    Event);                     // server-streaming
  rpc IngestEvents(stream Event) returns
    (IngestSummary);           // client-streaming
  rpc Chat(stream ChatMessage) returns (stream
    ChatMessage);              // bidi
}

message NotifyRequest {
```

```

    string tenant_id = 1;
    string subject = 2;
    string body = 3;
    string idempotency_key = 4;
}

message Event {
    string id = 1;
    string source = 2;
    string type = 3;
    google.protobuf.Timestamp time = 4;
    bytes data = 5;
}

```

§2.2 Wire format

gRPC over HTTP/2:

- HTTP/2 stream per RPC.
- Headers carry metadata (`grpc-encoding`, `authorization`, ...).
- Body carries length-prefixed protobuf frames.
- Trailers carry status (`grpc-status`, `grpc-message`).

Single message frame: [1 byte: compression flag][4 bytes: length BE][N bytes: protobuf body].

§2.3 Status codes

gRPC defines its own status codes (separate from HTTP):

- 0 OK
- 3 INVALID_ARGUMENT
- 5 NOT_FOUND
- 7 PERMISSION_DENIED
- 13 INTERNAL
- 16 UNAUTHENTICATED
- ... (see `grpc-status.h`)

§2.4 Deadline propagation

Client sets a deadline; server receives it via `grpc-timeout` metadata. Server-internal RPCs propagate the remaining deadline. Critical for fan-out workloads.

§2.5 Streaming flow control

HTTP/2 flow control applies. Server can send faster than client `recv` if the client's window allows.

§2.6 Compression

Per-message: `gzip` standard; `snappy` / `zstd` via plugins. Selected via `grpc-encoding` header.

§3. Herald-specific applicability analysis

§3.1 Where gRPC fits

Internal inter-flavor RPC — when pherald hands off events to sherald (safety classification) or cherald (compliance check), gRPC server-streaming saves bytes + connection overhead vs REST. NOT a public API.

High-rate event ingest — pherald exposes `IngestEvents(stream Event)` for systems that produce thousands of events/sec (Kubernetes audit logs, application logs). Each event is a few hundred bytes of protobuf vs a few KB of JSON.

Large data pulls — cherald exposes `StreamEvidence(stream Evidence)` for compliance evidence collection (potentially gigabytes; streaming avoids materializing everything in memory).

§3.2 Does gRPC replace REST?

No. REST stays as the primary external API (browsers, curl, third-party HTTP clients). gRPC is the INTERNAL high-throughput surface. Coexists.

§3.3 Multi-tenant + auth

JWT in `authorization` metadata; same `commons_auth/verifier`; same `tenant_id` claim extraction.

§3.4 Idempotency

For unary RPCs, idempotency key as a field in the protobuf message (e.g. `NotifyRequest.idempotency_key`). Same Redis SETNX path.

For streaming, each message in a stream carries its own idempotency key.

§3.5 OTel propagation

OTel's gRPC interceptor (go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc) handles traceparent propagation in gRPC metadata. Out-of-the-box.

§3.6 TLS

Reuse `commons_tls/.mTLS` for inter-flavor (strong service identity); 1-way TLS for external clients.

§3.7 Failure modes

1. **Streaming connection exhaustion.** Long-lived streams tie up connections. Limit concurrent streams per tenant.
2. **Backpressure mishandling.** If server emits faster than client consumes, HTTP/2 flow control kicks in but server-side goroutines may block. Use buffered channels + drop-oldest policy on slow consumers.

3. **Schema evolution.** Adding a field is safe; removing one breaks old clients. Strict review of .proto changes via mutation gate.

§4. Step-by-step implementation guide for Herald

§4.1 Add deps

google.golang.org/grpc, google.golang.org/protobuf. Both well-known, stdlib-quality; per spec V3 §9.1 these are exempt from the submodule requirement.

§4.2 New module commons_grpc/

```
commons_grpc/
├── go.mod
├── proto/                                # source-of-truth .proto files
│   ├── herald/v1/notifier.proto
│   ├── herald/v1/events.proto
│   ├── herald/v1/compliance.proto
│   └── herald/v1/safety.proto
├── gen/                                # generated code (committed; do NOT regen
│   └── herald/v1/*.pb.go
├── server/
│   ├── server.go                        # gRPC server constructor + interceptor w
│   ├── auth.go                          # JWT interceptor
│   ├── otel.go                          # OTel + tenant_id extraction
│   └── server_test.go
├── client/
│   ├── client.go                        # gRPC client wrapper + retries
│   └── client_test.go
└── e2e_real_grpcurl_test.go
```

§4.3 Protobuf compilation

scripts/grpc_gen.sh — runs buf generate or protoc per .proto file. Committed under gen/.

§4.4 Per-flavor service registration

Flavor	Service	RPCs
pherald	Notifier + EventIngest	Notify (unary), IngestEvents (client-stream), StreamEvents (server-stream)
cherald	Compliance	StreamEvidence (server-stream), Attest (unary)
sherald	Safety	StreamSafetyEvents (server-stream), QuerySafetyState (unary)
Others	(defer to Wave 4e+1)	

§4.5 New e2e_bluff_hunt invariants

- **E73** — grpcurl -d '{...}' :7104g herald.v1.Notifier/Notify → expected response.
- **E74** — server-streaming: client opens stream + receives N events.

- **E75** — bidi: client sends N + receives M; verifies independent streams.
- **E76** — deadline propagation: client deadline = 1s; server sleep = 2s; client RPC fails with DEADLINE_EXCEEDED.
- **E77** — JWT-less RPC → UNAUTHENTICATED.
- **E78** — schema break: modify .proto, regen, run against old client → expected new/old field handling.

§4.6 HRD scaffolding

- **HRD-276** — bootstrap commons_grpc/.
- **HRD-277** — .proto definitions.
- **HRD-278** — buf/protoc gen pipeline.
- **HRD-279** — server-side scaffolding + JWT interceptor.
- **HRD-280** — client-side wrapper.
- **HRD-281** — pherald Notifier service.
- **HRD-282** — cherald Compliance service.
- **HRD-283** — sherald Safety service.
- **HRD-284** — OTel propagation.
- **HRD-285** — TLS / mTLS.
- **HRD-286** — e2e_bluff_hunt E73–E78.
- **HRD-287** — mutation gates.
- **HRD-288** — spec V3 §4u amendments.
- **HRD-289** — operator runbook.
- **HRD-290** — close Wave 4e (gRPC slice).

§5. §107 anti-bluff testing strategy

The bar: grpcurl + real protobuf-typed Go client successfully invoke RPCs against running flavor binary.

§5.1 Happy-path tests

- **Test 1: grpcurl unary.** `grpcurl -d '{...}' :7104g herald.v1.Notifier/Notify` → JSON response. Physical proof: capture stdout + diff against golden.
- **Test 2: server-streaming.** `grpcurl -d '{...}' :7104g herald.v1.Notifier/StreamEvents` → N events received in order. Physical proof: log file shows N timestamps in increasing order.
- **Test 3: bidi.** Custom Go client sending 5 + receiving 5 independently.
- **Test 4: deadline.** Set client deadline; assert DEADLINE_EXCEEDED after the deadline.

§5.2 Edge cases

- **Test 5: oversized message.** `> grpc.MaxMsgSize` → RESOURCE_EXHAUSTED.
- **Test 6: backpressure.** Slow client; server's send blocks; assert no goroutine leak.

§5.3 Failure modes

- **Test 7: JWT-less.** UNAUTHENTICATED.
- **Test 8: wrong tenant.** Cross-tenant call → PERMISSION_DENIED.

§5.4 Performance

- **Test 9: throughput.** 10K events/sec ingest sustained for 60s.
- **Test 10: latency.** p99 < 10ms for unary.

§5.5 Mutation gates

- Mutation: remove JWT interceptor → Test 7 FAILS.
- Mutation: stub OTel interceptor → no `traceparent` propagation → trace-based test FAILS.

§5.6 Wire-level

- `tcpdump :7104g` → assert HTTP/2 frames; h2c decode; verify trailer `grpc-status: 0`.

§6. Open questions for operator

1. **Public gRPC or internal-only?** Recommend INTERNAL only — REST stays public face.
2. **Connect-go alongside gRPC for HTTP/1.1 compat?** Possibly for cases where a customer can't speak HTTP/2. Defer.
3. **gRPC-Gateway REST shim?** Auto-generates a REST handler from `.proto`. Could replace some hand-written REST handlers. Defer; evaluate after E73–E78 land.
4. **buf vs protoc?** Recommend buf (better lint + breaking-change detection).
5. **Commit generated .pb.go files?** Yes — avoid build-time codegen complexity.
6. **mTLS or 1-way?** mTLS for inter-flavor; 1-way for external.

§7. References

(All fetched 2026-05-22.)

- **gRPC docs.** <https://grpc.io/docs/languages/go/basics/>
- **gRPC Go.** <https://github.com/grpc/grpc-go>
- **gRPC core concepts.** <https://grpc.io/docs/what-is-grpc/core-concepts/>
- **Bidirectional streaming.** https://dev.to/yash_mahakal/implementing-bidirectional-grpc-streaming-a-practical-guide-3afi
- **buf.** <https://buf.build/>
- **OTel gRPC instrumentation.** <https://github.com/open-telemetry/opentelemetry-go-contrib/tree/main/instrumentation/google.golang.org/grpc/otelgrpc>

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no-match. Depend on `google.golang.org/grpc` + `google.golang.org/protobuf` via `go get` (per §9.1 carve-out for well-known stdlib-quality libs). No submodule needed.